

CSS Grid Basics

CSS Grid is a **powerful 2D layout system** in CSS that helps developers design web layouts using **rows and columns** easily. It provides more flexibility compared to older layout systems like **float** or **flexbox** (which is mainly one-dimensional).

What is a Grid?

A grid layout consists of:

- **Grid Container** → The parent element that holds the grid.
- **Grid Items** → The direct children of the grid container.
- **Grid Lines** → Horizontal and vertical dividing lines that form the grid structure.
- **Grid Tracks** → The spaces between grid lines (rows and columns).
- **Grid Cells** → The smallest individual rectangular space in the grid.
- **Grid Areas** → A combination of multiple grid cells.

Creating a Grid Container

You can define a grid by setting `display: grid;` or `display: inline-grid;` on a container.

```
.container {  
  display: grid;  
  grid-template-columns: 200px 200px 200px;  
  grid-template-rows: 150px 150px;  
  gap: 10px;  
}
```

This creates a grid of **3 columns** and **2 rows** with a **10px gap** between them.

Important Grid Properties

For Grid Container

Property	Description	Example
<code>display</code>	Defines the grid container.	<code>display: grid;</code>
<code>grid-template-columns</code>	Defines number and width of columns.	<code>grid-template-columns: 1fr 2fr 1fr;</code>

<code>grid-template-rows</code>	Defines number and height of rows.	<code>grid-template-rows: 100px 200px;</code>
<code>gap</code>	Sets the gap between rows and columns.	<code>gap: 20px;</code>
<code>justify-content</code>	Aligns grid items horizontally within the container.	<code>justify-content: center;</code>
<code>align-content</code>	Aligns grid items vertically within the container.	<code>align-content: space-between;</code>

For Grid Items

Property	Description	Example
<code>grid-column</code>	Defines how many columns an item spans.	<code>grid-column: 1 / 3;</code>
<code>grid-row</code>	Defines how many rows an item spans.	<code>grid-row: 1 / 2;</code>
<code>justify-self</code>	Aligns item horizontally within its cell.	<code>justify-self: center;</code>
<code>align-self</code>	Aligns item vertically within its cell.	<code>align-self: end;</code>

Example

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  grid-template-rows: 100px 100px;
  gap: 15px;
}

.item1 {
  background-color: lightcoral;
  grid-column: 1 / 3;
}
```

Explanation: The first grid item (`item1`) will span across the first two columns.

CSS Transformations

CSS transformations let you **manipulate elements visually** by rotating, scaling, skewing, or moving them. These transformations happen in **2D or 3D space**, without affecting other elements.

Common Transform Functions

Function	Description	Example
<code>rotate(angle)</code>	Rotates an element by the given angle.	<code>transform: rotate(45deg);</code>
<code>scale(x, y)</code>	Scales the size of an element.	<code>transform: scale(1.5, 1.5);</code>
<code>translate(x, y)</code>	Moves an element from its current position.	<code>transform: translate(50px, 100px);</code>
<code>skew(x-angle, y-angle)</code>	Skews (distorts) the element along X and Y axes.	<code>transform: skew(20deg, 10deg);</code>
<code>matrix(a, b, c, d, e, f)</code>	Combines multiple transformations in one.	<code>transform: matrix(1, 0, 0, 1, 30, 50);</code>

Transform Origin

Defines the point around which the transformation occurs.

```
transform-origin: center; /* default */
transform-origin: top left;
transform-origin: 50% 50%;
```

Example:

```
.box {
  transform: rotate(30deg);
  transform-origin: left top;
}
```

Here, the box rotates around its **top-left corner** instead of the center.

CSS Animations

CSS animations allow elements to **change their appearance or position gradually** using keyframes. You can create smooth transitions, moving effects, and visual feedback without JavaScript.

Key Concepts

1. **@keyframes** → Defines how the element changes during the animation.
2. **Animation Properties** → Controls the animation behavior (speed, timing, delay, etc.).

Basic Example

```
@keyframes colorChange {  
  0%   { background-color: red; left: 0; }  
  50%  { background-color: yellow; left: 50px; }  
  100% { background-color: green; left: 100px; }  
}  
  
.box {  
  position: relative;  
  width: 100px;  
  height: 100px;  
  animation: colorChange 3s ease-in-out infinite alternate;  
}
```

Animation Properties

Property	Description	Example
animation-name	Name of the keyframes animation.	animation-name: move;
animation-duration	Duration of the animation.	animation-duration: 4s;
animation-timing-function	Speed curve of the animation.	animation-timing-function: ease-in-out;
animation-delay	Time before the animation starts.	animation-delay: 2s;
animation-iteration-count	Number of times animation repeats.	animation-iteration-count: infinite;
animation-direction	Direction of animation (normal, reverse, alternate).	animation-direction: alternate;

Example with Hover Effect

```
.button {  
  background: royalblue;  
  transition: transform 0.5s;  
}  
  
.button:hover {  
  transform: scale(1.2) rotate(10deg);  
}
```

When you hover over the button, it smoothly **zooms in and rotates**.

CSS Frameworks

A **CSS framework** is a collection of pre-written CSS classes and reusable UI components that help speed up web development. These frameworks include **responsive grids**, **buttons**, **forms**, **navigation bars**, and more.

Popular CSS Frameworks

1. Bootstrap

- Most widely used CSS framework.
- Includes a 12-column responsive grid system.
- Comes with pre-built UI components like buttons, modals, and carousels.
- Uses both **CSS** and **JavaScript**.

Example:

```
<link rel="stylesheet"  
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.c  
ss">  
<button class="btn btn-primary">Click Me</button>
```

2. Tailwind CSS

- A **utility-first** framework.
- Instead of pre-made components, it provides low-level utility classes (e.g., `p-4`, `text-center`, `bg-blue-500`).
- Extremely customizable.

Example:

```
<button class="bg-blue-500 text-white px-4 py-2 rounded  
hover:bg-blue-600">Click</button>
```

3. Foundation

- Developed by ZURB.
- Known for its **mobile-first responsive grid**.
- Includes accessibility-focused UI components.

4. Bulma

- Built entirely on **Flexbox**.
- Simple and modern syntax.
- Provides readable class names like `.box`, `.button`, `.is-primary`.

5. Materialize

- Based on **Google's Material Design**.
- Provides ready-to-use components like cards, modals, and floating buttons.
- Great for modern UI design consistency.

Advantages of CSS Frameworks

- Faster web design and prototyping.
- Consistent and responsive layout.
- Cross-browser compatibility.
- Predefined color themes and components.

Disadvantages

- Extra unused CSS can increase file size.
- May limit design creativity.
- Learning curve for framework-specific syntax.